

**Wojciech Maj**

Posted on Dec 20, 2021 • Updated on May 17

Enzyme is dead. Now what?

#react #testing

I'm the maintainer of `@wojtekmaj/enzyme-adapter-react-17`. **I'm here to warn you. Enzyme**, a popular utility to test React components, **is dead**. It's time to move on. Here's why I think so.

For a long time, Enzyme has been the 1st choice when it comes to testing React applications, despite of how bumpy the road to Enzyme React 16 support was. Even today, [1/3 of React apps are still being tested using Enzyme!](#)

How `@wojtekmaj/enzyme-adapter-react-17` came to life

Back in August 2020, React 17 Release Candidate [came out](#). Shortly after, an [issue has been raised](#) in Enzyme repository to add support for React 17. Immediately after, [@layershifter](#) has [opened a PR](#) adding an official `enzyme-adapter-react-17`.

Unfortunately, there were (and, spoiler alert, still are) some issues with testing that prevented this PR from being merged.

Now, here's the thing. I'm the maintainer of many popular React packages, [React-PDF](#), [React-Calendar](#), and [React-Date-Picker](#) just to name a few. Professionally, I maintain several large projects, which collectively have more than 30,000 Enzyme-based unit tests.

I was in no position to wait for Enzyme to add support for React 17 to support React 17 in my packages. Rewriting all unit tests also looked like no fun to me. So, I've decided to act and publish a temporary adapter based on Oleksandr's work.

It wasn't only the adapter though: certain changes were also necessary in Enzyme adapter utils, which all Enzyme adapters depend on, so a fork of them was also needed. A couple of tireless evenings later, `@wojtekmaj/enzyme-adapter-react-17` was born.

Fast-forward one year

The issue for adding React 17 support remains open and has become nothing but a source of bitter comments mixed with unproductive "any updates?". Oleksandr's PR remains unmerged. `enzyme-adapter-react-17` is nowhere in sight. **The unofficial adapter I published** with an intention of being just a stepping stone before everyone eventually migrates to `enzyme-adapter-react-17`, **has become *de facto* the default adapter for React 17**, with 16 million downloads so far.

Retrospectively, **I'm not sure if publishing the package was a good decision.**

On one hand, this move has helped thousands of developers worldwide to upgrade to React 17, relatively hassle-free.

On the other, I helped prolonging Enzyme's inevitable death and thus gave many developers hope and a sense of safety they shouldn't get to feel. **And I'm sorry for that.**

React 18

Before I came to this conclusion though, React 18 beta was announced, and of course, I jumped to see if `@wojtekmaj/enzyme-adapter-react-18` **could be released.**

The big thing in React 18 are concurrent features. To opt-in, after upgrading to React 18, you also need to switch to the new `createRoot` API:

```
// before
const container = document.getElementById('root');
ReactDOM.render(<App />, container);

// after
const container = document.getElementById('root');
const root = ReactDOM.createRoot(container);
root.render(<App/>);
```

Until you switch to the new API, your app will behave as if it's running React 17. So if you don't plan to do this, there's no point of upgrading!

I quickly realized that the API changes in React 18 meant that **releasing a React 18 Enzyme adapter will not be possible** without a huge rework of not only the adapter itself, but also Enzyme, `enzyme-adapter-utils` and `enzyme-adapter-react-helper`.

Trust me, it's not gonna happen. It's over. No more "stepping stones". And certainly no more official adapters. **Whether or not you plan to upgrade to React 18 in the near**

more general adapters. Whether or not you plan to upgrade to React 18 in the near future, you should consider looking for Enzyme alternative right now.

What should I do? 🤖

The answer is, as always, it depends. You don't *have* to upgrade React, after all.

Here's what I would do:

- Start [familiarizing yourself with React Testing Library](#), an officially recommended library for React components.
- Make a rule to write new tests using RTL only.
- Consider making a rule to rewrite tests to RTL whenever you need to touch them and/or the component they are testing.
- In your designated time for repaying technical debt (you have designated time for repaying technical debt, right? ...right?), rewrite your remaining Enzyme-based tests to RTL.
- Clean up your repo from Enzyme-specific bits
- When you're ready, upgrade to React 18.

A bit of personal advice

While [Migrate from Enzyme support article](#) is available, I suggest you to just start fresh, forgetting that Enzyme has ever existed. **RTL is by no means an Enzyme drop-in replacement**, so having a completely fresh mindset will help you getting the most of it.

Discussion (13)



Andrew McCallum • Dec 20 '21



RTL is by no means an Enzyme drop-in replacement, so having a completely fresh mindset will help you getting the most of it.

You make a good point. RTL is more of an integration testing library where as Enzyme has the ability to unit test components. This move from unit to integration test focus is a pretty big change for some applications.



Marabesi • Jan 23



Even though I see how the thinking behind integration x unit test is, I would recommend

to watch Mario's talk on fragile tests (blog.devgenius.io/tdd-conference-2...).

The mindset of unit x integration around RTL is something that comes up often, and, it's possible to test a single component with RTL as well. The point is, this is not the philosophy of the library (testing-library.com/docs/guiding-p...).

In this sense, what I understand is: unit does not mean 1-1 (one test one component) rather, one behavior that you want to test.



Marco Colli • Dec 21 '21



Call me old-fashioned by in 10+ years of development with Rails I never seen a breaking change like this and I never had to rewrite an entire test suite... That is why I always fear the adoption of new trends.



Sergio Daniel Xalambri • Dec 22 '21



I think the issue was because Enzyme dependent on internals of React, while tools like RTL test them from the outside, in Rails you have something similar, the test frameworks like RSpec are not dependent on Rails internals to work.

Enzyme did a bad thing there, RTL got it right, so right that testing-library has now a generic testing-library-dom and wrappers for different frameworks, theoretically a test for React could work with only a few changes to test also a Vue component, after the test render searching dom nodes and triggering events is the same.



Attila Szeremi ⚡ • Dec 23 '21



I don't think that's a good excuse. Laravel gives you testing tools that interact with the internals of Laravel, and there are hardly ever any breaking changes there.

True though that the Laravel testing tools were created by the creators of Laravel and is officially endorsed, whereas Enzyme is completely unofficial and 3rd party to React.



Mikael Gramont • Jan 18



Agreed. That FB doesn't really offer a testing solution is the problem here: people set out to build their own test libraries while FB was free to break them.



André Matulionis • Dec 22 '21



FYI, publishing a unofficial package was a savior for me. I had to upgrade NextJS in a huge repo for vulnerability issues, which also upgraded React to v17

All tests for the last 3 years were done using rtl. But there is a considerable part of old tests that still are written using Enzyme. If it weren't for the adapter, the vulnerability issue would become a huge test-rewriting task.

Also, thanks for the warning. Will add the migration to rtl to our roadmap



Piotr Staniów • Jan 5 • Edited on Jan 5



Thanks for the great work on the adapter! I think we've all seen it coming for last couple of years now, given how long it took to get support for React 16.x. In fact, it still doesn't work in various scenarios, nonetheless it's a painful process to move away from it so we all wanted to avoid it.

I like how Facebook tackles it, keeping the old Enzyme tests alive but writing the new ones only in React Testing Library, and gradually upgrading the codebase when touched.

I wrote an article a few months ago ([Time to say goodbye - Enzyme.js](#)) to suggest that it's time to call the library deprecated, as it would help to convey the point about making such migration across companies. Alas, the idea didn't resonate with the last maintainer of Enzyme.

Maybe eventually the React core team will make that call on their docs page, that they discourage using it, who knows...



tieje • May 17



This is so crazy.



Alex Lohr • Dec 21 '21



I came here expecting a rant and was surprised by genuinely interesting content. I had already moved on to RTL in the meantime, but still wondered what had happened. Thanks for satiating my curiosity.



Kumar Chetan Sharma • Feb 3



you have designated time for repaying technical debt, right? ...right?

(☹_☹) We have quite a few things on Enzyme. Gotta explain to product managers where the sprint bandwidth is going cause we don't have a designated time for repaying tech debts.

Everyone has broken up with Enzyme at work... RTL is the new "bae".



Lars Ejaas • Dec 21 '21



I also really keep my eyes on the new Vitest library. It is in beta, and not ready for production yet. But it looks promising at least! vitest.dev

[Code of Conduct](#) • [Report abuse](#)



Wojciech Maj

LOCATION

Kraków, Poland

WORK

CTO at Rewardo

JOINED

Mar 24, 2017

More from Wojciech Maj

Migrating your React app from Webpack to Vite

[#javascript](#) [#react](#) [#vite](#) [#tutorial](#)

Optimizing React apps: Hardcore edition

[#javascript](#) [#react](#) [#webpack](#) [#tutorial](#)

How to enable new JSX transform in React?

[#javascript](#) [#react](#) [#babel](#) [#tutorial](#)